

Security Design Document

Secure Document Sharing System (SDSS)

CS 419: Secure Web Application Project

Team: Alina Nesimi & Mina Odabasi

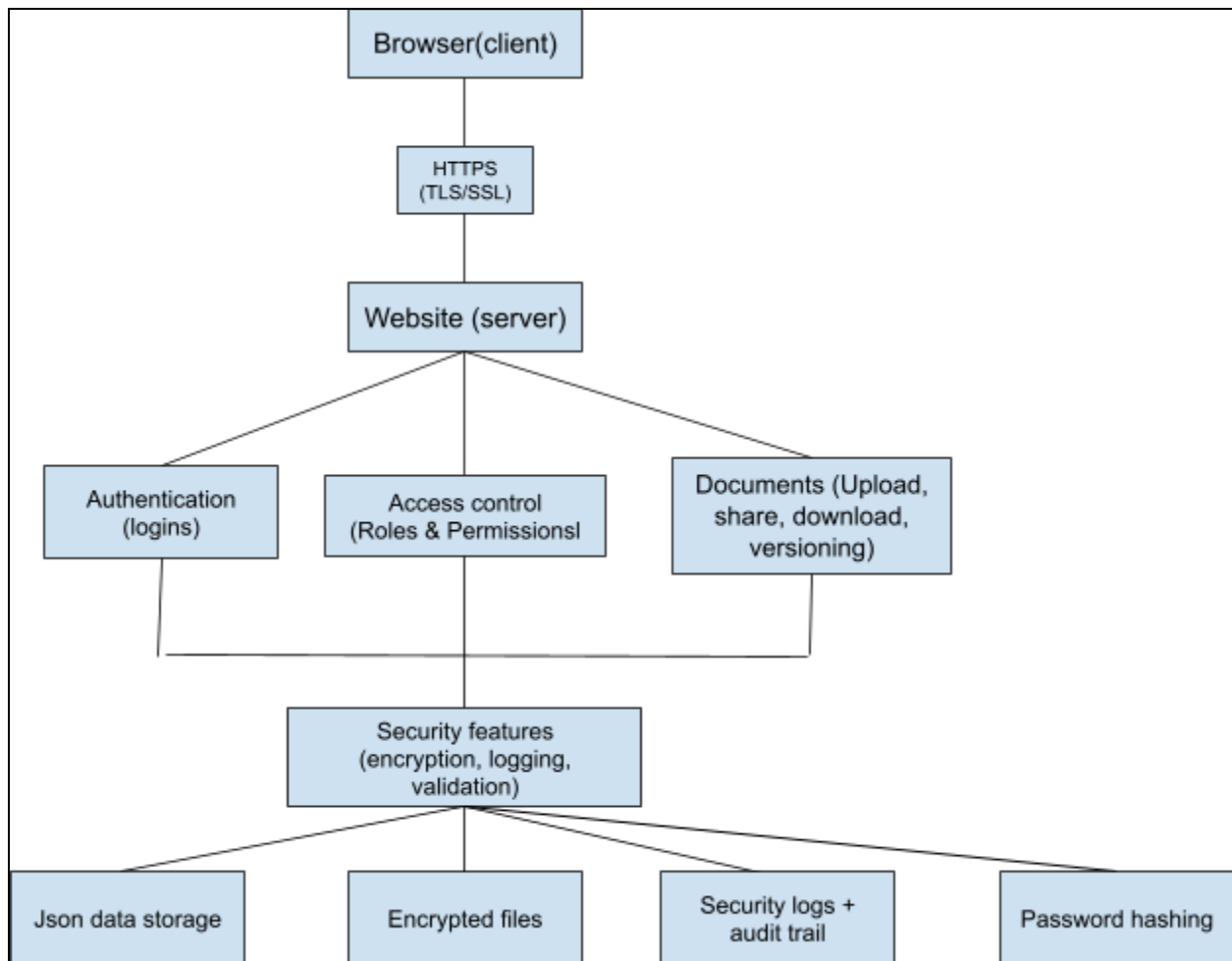
Introduction

The Secure Document Sharing System is an application built on the Flask framework. This application offers a secure platform for users to upload, save, share and manage private documents. It features user authentication, role based access control, encrypted document storage, sharing permissions, document version control, an audit trail, and administrative management.

This system is primarily focused on maintaining document confidentiality, integrity, availability, and traceability. To address the problem, the system design employs a variety of methods that include, but are not limited to, password hashing, session control, control access and validations, limitation of the input of control lengths, and validations, and control logging. This document describes the different security control measures adopted, the system's architecture and design, threat model, data flow and the measures taken to protect the integrity of the system.

A. Architecture Overview

1. System Architecture Diagram



2. Data Flow Diagram

Login Flow

User → Login Request → Validate → Check credentials (email, username, password) → Create the session → store Session → return cookie → user has been authenticated

File Upload Flow

User → Login → Upload File → Validate file → Encrypt file → Store Encrypted File

File Download Flow

User → login → Authenticate → Authorize access/permission check → get encrypted file → decrypt → DOWNLOAD

3. Component Descriptions

1. Browser (client)

- Sends HTTP/HTTPS requests
- Displays UI (login, upload, download)
- No direct security logic (frontend only reflects backend)

2. Python Flask Backen

- This is the core of the applications logic since it handles authentication and authorization (RBAC / access control) and processing of files (upload, dow

3. Security features

- File encryption
- Password hashing
- Role-based access control
- HTTPS

4. Json Data storage

- This stores data inside of json file which includes:
 - Users
 - Password hashes
 - Sessions
 - Who doc is shared with
 - User role
 - File metadata (NOT file content)

4. Logs

- Security logs
- Audit Logs

4. Technology Stack Justification

- Python (Flask)
 - Simple and lightweight framework
 - Easy to build web applications quickly
 - Supports authentication and security features

- JSON Data Storage
 - Easy to use and requires no database setup
 - Good for storing users, sessions, and document metadata
 - Suitable for small-scale applications
- bcrypt (Password Hashing)
 - Secure way to store passwords
 - Prevents passwords from being saved in plain text
- Fernet Encryption
 - Encrypts files before storing them
 - Protects sensitive document data
- HTTPS (TLS/SSL)
 - Encrypts communication between user and server
 - Prevents interception of data (e.g., passwords)

B. Threat Model

1. Asset Identification

- User credentials
- Uploaded files
- Encryption keys
- Database information
- Key.pem

- Session cookies

2. Threat Enumeration

- **External threats**
 - Unauthorized access
 - Brute force attempts
 - Man-in-the-middle attacks
 - File interception
- **Internal threats**
 - Privilege escalation
- **Application Threats**
 - XSS (Cross-site scripting)
 - File upload attacks

3. Vulnerability Assessment

- If you have weak storage of passwords → passwords can be leaked
- If file not encrypted → file could be accessed by hacked
- No access control → unauthorized file access
- No file type spoofing → malware can be uploaded

4. Attack Scenarios

- **Scenario 1: Network Sniffing**

- Attacker intercepts traffic → HTTPS encrypts data → therefore unreadable data
- **Scenario 2: Malicious File Upload**
 - User uploads disguised file renames extension to .pdf
 - Validation (MIME) prevents unauthorized file types
- **Scenario 3: Stolen Database**
 - Attacker gains DB
 - Passwords unreadable because of bcrypt (hash)

5. Risk Prioritization

- Malicious File content/MITM attack is a high severity
- Credential theft - high severity
- File viewability - high severity
- XSS - medium severity

C. Security Controls

1. Authentication Control

- **Description:**
 - Make sure users that are registered can access the website by requiring a valid username and password.

- **Implementation:**

- Require complex password (12 char, uppercase, number, special char)
 - Uses bcrypt hashing algo to securely store passwords

- **Testing:**

- Attempt login with incorrect credentials

- **Limitations:**

- Users may reuse passwords from different sites
- No Multi factor authentication

- **Mitigation:**

- Enforce MFA
- Enforce unique password usage

2. Authorization (Role Based Access Control)

- **Description:**

- Restricts access to features/actions and documents based on user roles and permissions

- **Implementation:**

- Enforce checks to see if user is logged in and if their role allows them to perform said actions
- Also we used wrapper / decorators to enforce access control:
 - @require_auth

- `@require_role("admin")`

- **Testing:**

- Attempt to access admin routes as a normal user
- Verify restricted actions are blocked

- **Limitations:**

- Need to be careful with role assignment

- **Mitigation:**

- Consistent role handling

3. File Encryption

- **Description:**

- Protects sensitive documents by encrypting files before storing them

- **Implementation:**

- Files are encrypted before storage for ex:
 - `encrypted_data = cipher.encrypt(file_bytes)`

- **Testing:**

- When you open stored files directly they should be encrypted and unreadable

- **Limitations:**

- Encryption key must be securely managed

4. File Validation

- **Description:**

- Prevents malicious files from being uploaded

- **Implementation:**

- Enforces file extension whitelisting
- Validates file content using MIME

- **Testing:**

- Attempted to upload file type that was not allowed

- **Limitations:**

- Advanced file techniques could bypass this

- **Mitigation:**

- Use more complex file examinations

5. HTTPS

- **Description:**

- Provides secure communication between client & server using encryption

- **Implementation:**

- enabled using SSL context:

- `app.run(ssl_context=("cert.pem", "key.pem"))`

- **Testing:**

- Attempt to access via HTTP and redirected to HTTPS

- **Limitations:**

- Self-signed certificates trigger browser warnings

- **Mitigation:**

- Use trusted certificate authority (CA) in actual app production

D. Data Protection

1. Data Classification

- a. Passwords - high sensitivity
- b. Files - high sensitivity
- c. Metadata (json) - medium sensitivity
- d. Logs - low sensitivity

2. Encryption Methods

- a. AES (Fernet) used to encrypt files
- b. HTTPS (SSL/TLS)

3. Key Management

- a. Keys generated securely
- b. Never hardcoded

4. Secure Deletion Procedures

- a. Delete file from upload folder
- b. Delete file info from json storage
- c. Overwrite file

Conclusion:

The Secure Document Sharing System was designed with multiple layers of security to protect confidential documents and user accounts. The application combines authentication protections, role based access control, encrypted storage, secure sessions, input validation, logging, audit trails, and security headers in one system.

This design reflects the secure web development principles expected for the project. While a few limitations remain, the application demonstrates strong effort in protecting confidentiality, integrity, accountability, and controlled access to data. Within the scope of the assignment, the system provides a strong example of secure by design web application development.